Solution: Combination Sum

Problem Statement

Given an array of distinct positive integers `candidates` and a target integer `target`, return a list of all **unique combinations** of candidates where the chosen numbers sum to `target`. You may return the combinations in **any order**.

The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

Example 1:

```
Input: candidates = [2, 3, 6, 7], target = 7
Output: [[2, 2, 3], [7]]
Explanation: The elements in these two combinations sum up to 7.
```

Example 2:

```
Input: candidates = [2, 4, 6, 8], target = 10
Output: [[2,2,2,2,2], [2,2,2,4], [2,2,6], [2,4,4], [2,8], [4,6]]
Explanation: The elements in these six combinations sum up to 10.
```

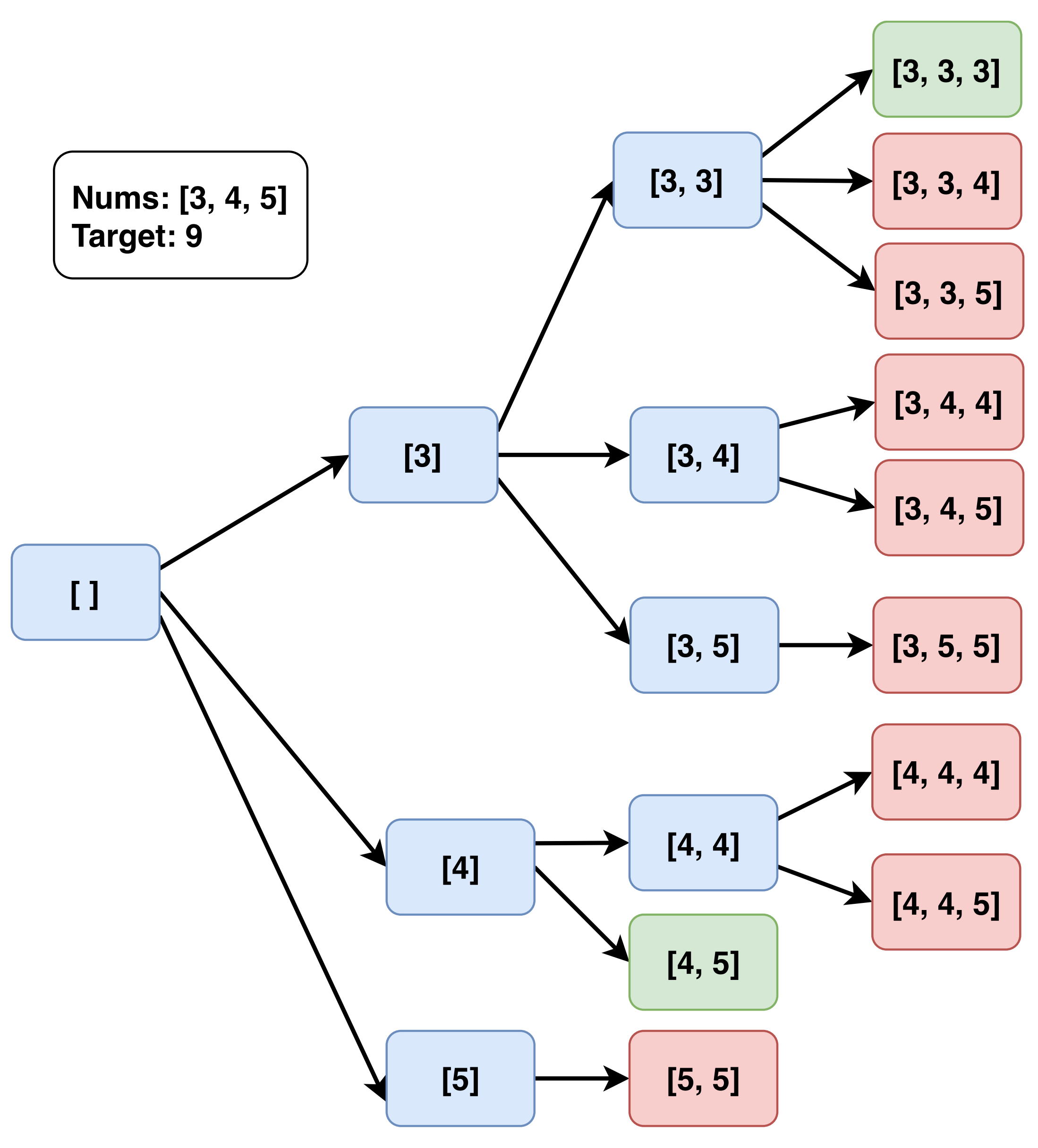
Solution

This problem is quite similar to the example discussed in the previous lesson (placing apple, orange, and mango trees).

We can follow the brute-force approach to incrementally build the solution.

If, at any time, we find out that the current solution can't lead to a valid combination, we will abandon it and backtrack to try the remaining solutions.

Let's try this approach on the following input: **Candidates**: [3, 4, 5], **Target**: 9



Here are the number of steps in our algorithm:

1. We will start with an empty set. This also means that the sum of the elements in the set is zero.
2. We can try adding all three numbers separately in the set. This will give us three sets: [3], [4], [5].
3. Let's take set [3], since the sum of its elements is less than the Target (=9), we will try adding more numbers to it.
4. We can add all three numbers again to generate new sets. We can add '3' again, as each number can be added multiple times.
5. After adding all numbers separately to the set [3], we will get the following sets: [3, 3], [3, 4], [3, 5]. We can see, each set has a sum less than the target.
6. We can now, repeat the same process as described in step-4.
    - For [3, 3]: Adding '3' will give us a valid solution [3, 3, 3] having a sum equal to the target. Adding '4' and '5' will give us a sum which is greater than the target. Therefore, we can stop the search here for this set, as adding any additional number will not produce a correct solution.
    - For [3, 4]: We will add '4' or '5' to it, which will give us a sum greater than the target. We will stop our search here for this set.
    - For [3, 5]: We can only add '5' to it, which does not produce a valid solution.
7. Similar approach can be applied to other sets.
8. In the end, we can see that the valid solutions are: [3, 3, 3] and [4, 5].

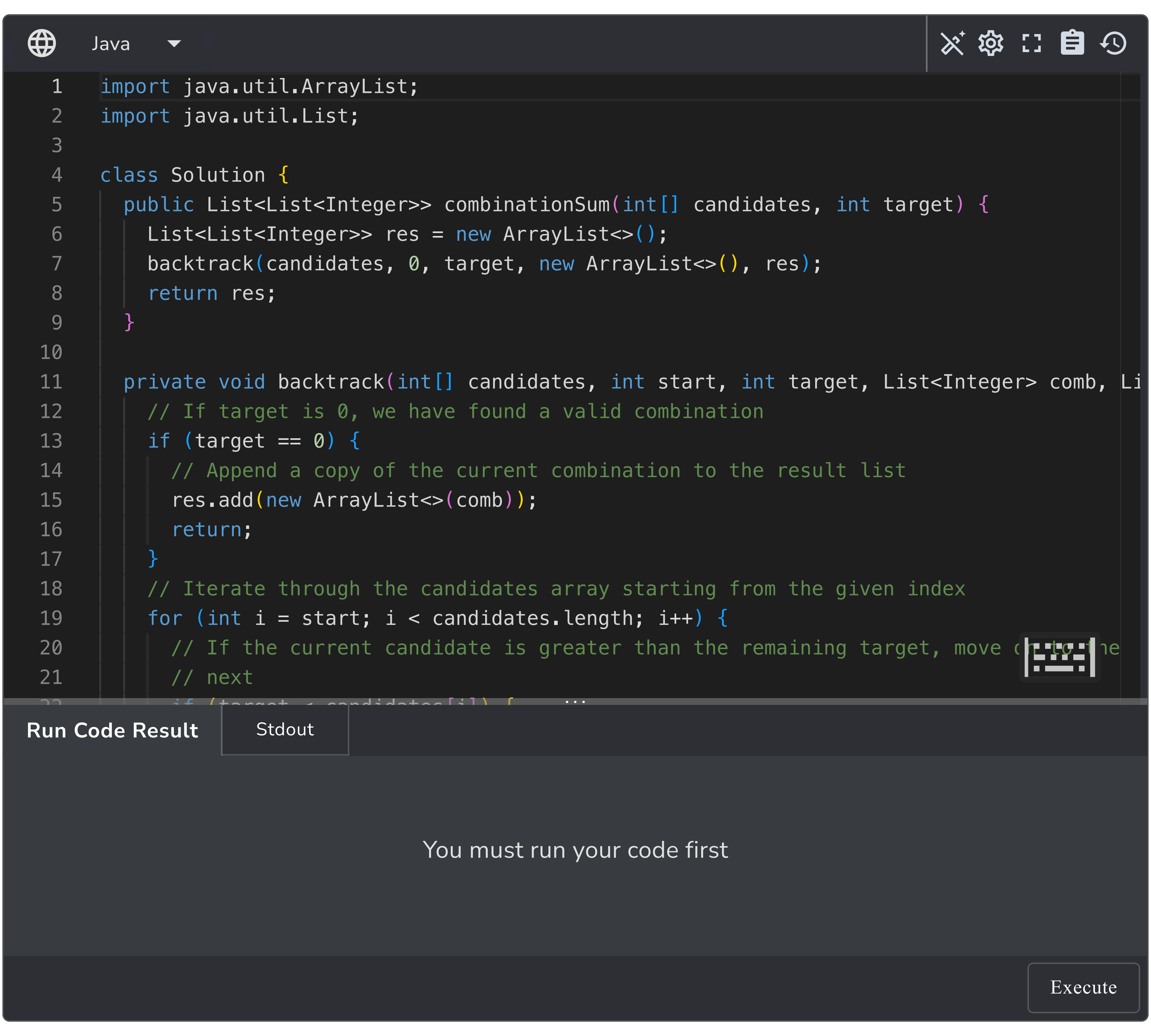
Code

The basic idea is to start with an empty combination and iterate through the candidates array, adding each candidate to the current combination and recursively calling the function with the updated combination and the remaining target. If the target becomes 0, we add the current combination to the result list. If the target becomes negative, we backtrack and remove the last added candidate from the combination.

The function `combinationSum` takes in two parameters, an array of distinct integers `candidates` and a target integer `target`, and returns a list of all unique combinations of candidates where the chosen numbers sum to target. The function starts by defining the function `backtrack(candidates, start, target, comb, res)`. This function takes five parameters, `candidates`, `start`, `target`, `comb`, and `res`:

- `candidates` is the array containing candidate elements.
- `start` is the starting index of the `candidates` array.
- `target` is the remaining target.
- `comb` is the current combination.
- `res` is the final result list.

The `backtrack` function uses recursion to find the combinations. The base case for the recursion is if the `target` is 0. When the `target` is 0, that means we have found a valid combination and we append a copy of the current combination to the result list. It then iterates through the candidates array starting from the given index. If the current candidate is greater than the remaining target, it skips the current candidate and move on to the next. If the current candidate is less than the remaining target, it adds the current candidate to the current combination, recursively calls the function with the updated combination and remaining target, and then backtracks by removing the last added candidate from the combination.



```java
import java.util.ArrayList;
import java.util.List;

class Solution {
  public List<List<Integer>> combinationSum(int[] candidates, int target) {
    List<List<Integer>> res = new ArrayList<>();
    backtrack(candidates, 0, target, new ArrayList<>(), res);
    return res;
  }

  private void backtrack(int[] candidates, int start, int target, List<Integer> comb, Li
    // If target is 0, we have found a valid combination
    if (target == 0) {
      // Append a copy of the current combination to the result list
      res.add(new ArrayList<>(comb));
      return;
    }
    // Iterate through the candidates array starting from the given index
    for (int i = start; i < candidates.length; i++) {
      // If the current candidate is greater than the remaining target, move on to the
      // next
```

You must run your code first

Time Complexity

This algorithm has a time complexity of $O(N^{T/M+1})$, where N is the total number of elements in the candidates array, T is the target value, and M is the smallest value among the candidates. This is because the execution of the backtracking is similar to a DFS traversal of an n-ary tree. So, the time complexity would be the same as the number of nodes in the n-ary tree. This can be seen in the above diagram.

Each node can call the `backtrack` function a maximum of N times, i.e., the total number of candidates. The maximal depth of the n-ary tree would be T/M, where we keep on adding the smallest element to the combination. As we know, the maximal number of nodes in N-ary tree of T/M height would be $N^{T/M+1}$, hence the time complexity is $O(N^{T/M+1})$.

Space Complexity

Ignoring the space needed for the output array, the space complexity will be $O(T/M)$ because at any time, we can pile up to T/M recursive calls of the `backtrack` function; this will happen when we keep on adding the smallest element to the combination. As a result, the space overhead of the recursion is $O(T/M)$.

← Previous
Combination Sum (Medium)

Next →
Word Search (Medium)

☐ Mark as Completed